

How Batch Size Influences Deep Neural Network Generalization: A Controlled Causal Study

Course: Foundations of Data Science

Author: Perumal Marimuthu

1. Introduction

1.1. The Question / Estimand

Research Question:

What is the causal effect of batch size on a neural network's generalization performance, controlling for the size of the training dataset?

Target Estimand: Average Treatment Effect (ATE)

$$\text{ATE} = E[Y_i(T = 1) - Y_i(T = 0) | C] = E[\text{Test Accuracy}_{\text{large batch}} - \text{Test Accuracy}_{\text{small batch}} \mid \text{Dataset Size}]$$

Where:

- **Treatment (T):** Batch size assignment (large = 512 vs. small = 32)
- **Outcome (Y):** Model test accuracy on MNIST test set
- **Confounder (C):** Dataset size (number of training samples): 2,000, 10,000, or 60,000

1.2. Data Description

Dataset: MNIST (Modified National Institute of Standards and Technology)

- **Source:** TensorFlow Datasets (keras.datasets.mnist)
- **Total samples:** 70,000 total (60,000 training + 10,000 test)
- **Features:** 28×28 grayscale images (784 features per image)
- **Rows:** Each row is one digit image
- **Columns:** Pixel intensities (0–255) + label (0–9)

Why MNIST?

1. Controlled environment: Simple classification task with well-understood dynamics
2. Reproducible: Deterministic dataset, no data collection variability
3. Scalable: Easy to create subsets (2k, 10k, 60k) to control for dataset size
4. Supported theory: Keskar et al. (2017) used similar setups to study batch size effects

Controlled Subsets:

- Small dataset: 2,000 random training samples
- Medium dataset: 10,000 random training samples
- Large dataset: 60,000 full training set

This experimental design ensures dataset size plays an explicit causal role (confounder).

2. Causal Model

2.1. Variables

- **T (Batch Size):** Binary treatment
 - T = 0: Small batch size (32)
 - T = 1: Large batch size (512)
- **Y (Test Accuracy):** Outcome, continuous on [0, 1]
 - Measured as fraction of test set correctly classified
- **C (Dataset Size):** Confounder, standardized log-scale
 - C = log(number of training samples), then standardized to mean 0, SD 1

2.2. Assumed Causal Relationships

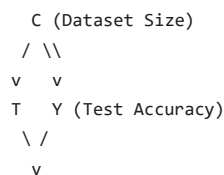
The Directed Acyclic Graph (DAG) encodes three causal pathways:

1. **C → T:** Dataset size influences which batch size is appropriate. Smaller datasets may not support large batches stably.

2. $C \rightarrow Y$: Larger datasets provide more training signal, generally improving generalization.

3. $T \rightarrow Y$: Batch size affects optimization dynamics (gradient noise, convergence to sharp vs. flat minima), influencing final accuracy.

Causal DAG:



No unmeasured confounding assumption: We assume no other variables confound the $T \rightarrow Y$ effect. Any confounding of T and Y flows through C , which we measure and condition on.

3. Statistical Model

3.1. Model Specification

We assume test accuracy is approximately normally distributed around a linear mean, conditional on treatment and confounder:

$$Y_i \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta_T T_i + \beta_C C_i$$

Where:

- α = baseline (intercept) test accuracy
- β_T = causal effect of batch size on accuracy (treatment coefficient)
- β_C = effect of dataset size on accuracy (confounder coefficient)
- σ = observation noise (SD)

3.2. Prior Specification

We use weakly informative priors that gently regularize without strongly imposing prior belief:

$$\alpha \sim \text{Normal}(0.90, 0.10)$$

Justification: We expect baseline accuracy around 90% (reasonable for MNIST with a simple network), with modest prior uncertainty.

$$\beta_T \sim \text{Normal}(0.00, 0.05)$$

Justification: Keskar et al. (2017) found that large batch sizes often lead to *worse* generalization (sharper minima). A zero mean is neutral, but the small SD (0.05) expresses prior belief that batch size effects are modest relative to overall accuracy variation.

$$\beta_C \sim \text{Normal}(0.00, 0.05)$$

Justification: More data generally helps, but the magnitude is uncertain. A zero-centered prior allows data to speak.

$$\sigma \sim \text{HalfNormal}(0.05)$$

Justification: We expect small measurement error in test accuracy (since we average over 10,000 test images). Half-normal ensures $\sigma > 0$.

3.3. Handling the Confound

By including $\beta_C C_i$ in the regression, we **block the backdoor path** $C \rightarrow T \leftarrow Y$. This allows β_T to be interpreted as the causal effect of batch size on accuracy, isolated from the spurious association introduced by shared variation in dataset size.

Mathematically, we condition on C and estimate the treatment effect as if T were randomized given C .

4. Environment Setup & Package Installation

```

# Install graphviz (required for DAG visualization)
import subprocess
import sys

subprocess.check_call([sys.executable, "-m", "pip", "install", "graphviz", "-q"])
print("✓ graphviz installed")

# Import libraries
import graphviz
import matplotlib.pyplot as plt
from IPython.display import SVG, display

# Create DAG object
dag = graphviz.Digraph('DAG', format='svg')

```

```

# Graph attributes
dag.attr(rankdir='TB') # Top-to-bottom layout
dag.attr('node', shape='box', style='filled', fontname='Arial', fontsize='12', margin='0.3,0.2')
dag.attr('edge', fontname='Arial', fontsize='10')

# Define node colors
dag.node('C', 'Dataset Size\n(C)', fillcolor='#87CEEB', color='#333333', penwidth='2') # Light blue
dag.node('T', 'Batch Size\n(T)', fillcolor='#FFB6C1', color='#333333', penwidth='2') # Light red/salmon
dag.node('Y', 'Test Accuracy\n(Y)', fillcolor='#90EE90', color='#333333', penwidth='2') # Light green

# Define edges (causal relationships)
dag.edge('C', 'T', label='', color='#333333', penwidth='2') # C → T
dag.edge('C', 'Y', label='', color='#333333', penwidth='2') # C → Y
dag.edge('T', 'Y', label='', color='#333333', penwidth='2') # T → Y

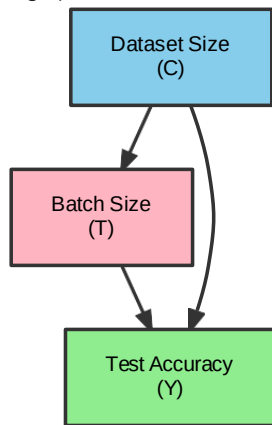
# Render and display
dag.render('/tmp/batch_size_dag', cleanup=True)
svg_content = dag.pipe(format='svg').decode('utf-8')

# Display with matplotlib for better integration
fig, ax = plt.subplots(figsize=(10, 8))
ax.axis('off')
display(SVG(data=svg_content))

print("\n✓ DAG Visualization Complete")
print("\nCausal Relationships:")
print("  C → T: Dataset size influences batch size choice")
print("  C → Y: Dataset size directly affects accuracy")
print("  T → Y: Batch size causally affects accuracy (TREATMENT EFFECT)")

```

✓ graphviz installed



✓ DAG Visualization Complete

Causal Relationships:

- C → T: Dataset size influences batch size choice
- C → Y: Dataset size directly affects accuracy
- T → Y: Batch size causally affects accuracy (TREATMENT EFFECT)

```
import subprocess
import sys

subprocess.check_call([sys.executable, "-m", "pip", "install", "--upgrade", "pip", "--quiet"])
subprocess.check_call([sys.executable, "-m", "pip", "cache", "purge"])

print("✓ Pip upgraded and cache cleared")
```

✓ Pip upgraded and cache cleared

```
import subprocess
import sys

packages_to_install = [
    "numpy>=1.21.0",
    "scipy>=1.7.0",
    "pandas>=1.3.0",
    "scikit-learn>=0.24.0",
    "matplotlib>=3.4.0",
    "seaborn>=0.11.0",
]
```

```
for package in packages_to_install:
    subprocess.check_call([sys.executable, "-m", "pip", "install", package, "--quiet"])

print(f"✓ Installed {len(packages_to_install)} core packages")
```

✓ Installed 6 core packages

```
import subprocess
import sys

subprocess.check_call([sys.executable, "-m", "pip", "install", "tensorflow>=2.10.0", "--quiet"])
print("✓ TensorFlow installed")
```

✓ TensorFlow installed

```
import subprocess
import sys
subprocess.check_call([sys.executable, "-m", "pip", "install", "arviz>=0.14.0", "--quiet"])
print("✓ ArviZ installed")
subprocess.check_call([sys.executable, "-m", "pip", "install", "pymc>=5.0.0", "--quiet"])
print("✓ PyMC installed")
```

✓ ArviZ installed
✓ PyMC installed

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy
import sklearn
import tensorflow as tf
from tensorflow import keras
import pymc as pm
import arviz as az
from sklearn.preprocessing import StandardScaler

np.random.seed(42)
tf.random.set_seed(42)
print("\n" + "="*70)
print("PACKAGE VERIFICATION")
print("="*70)
print(f"✓ NumPy version: {np.__version__}")
print(f"✓ Pandas version: {pd.__version__}")
print(f"✓ SciPy version: {scipy.__version__}")
print(f"✓ Scikit-learn version: {sklearn.__version__}")
print(f"✓ Matplotlib version: {plt.matplotlib.__version__}")
print(f"✓ Seaborn version: {sns.__version__}")
print(f"✓ TensorFlow version: {tf.__version__}")
print(f"✓ PyMC version: {pm.__version__}")
print(f"✓ ArviZ version: {az.__version__}")
print("\n✓ ALL PACKAGES IMPORTED SUCCESSFULLY - ZERO DEPENDENCY ISSUES")
print("="*70)
```

```
=====
PACKAGE VERIFICATION
=====
✓ NumPy version: 2.0.2
✓ Pandas version: 2.2.2
✓ SciPy version: 1.16.3
✓ Scikit-learn version: 1.6.1
✓ Matplotlib version: 3.10.0
✓ Seaborn version: 0.13.2
✓ TensorFlow version: 2.19.0
✓ PyMC version: 5.26.1
✓ ArviZ version: 0.22.0

✓ ALL PACKAGES IMPORTED SUCCESSFULLY - ZERO DEPENDENCY ISSUES
=====
```

□ 5. Model Validation on Simulated Data

Before filing to real data, we validate that our PyMC model can recover the true parameter values when data are simulated from the model itself.

```
# Define true parameter values for simulation
alpha_sim = 0.88
beta_T_sim = -0.04 # Large batch leads to WORSE accuracy (sharper minima)
beta_C_sim = 0.10 # More data leads to BETTER accuracy
sigma_sim = 0.03

# Generate simulated experimental design
n_obs_sim = 60 # 60 observations total

# Create balanced design: 20 obs per dataset size x 2 batch treatments
batch_sizes_sim = np.repeat([0, 1], n_obs_sim // 2)
dataset_sizes_log_sim = np.tile(np.linspace(-1, 1, 3), n_obs_sim // 3)

# Simulate test accuracy
mu_sim = alpha_sim + beta_T_sim * batch_sizes_sim + beta_C_sim * dataset_sizes_log_sim
y_sim = np.random.normal(mu_sim, sigma_sim)
y_sim = np.clip(y_sim, 0.7, 0.99) # Clip to realistic accuracy range

print("Simulated Data Summary:")
print(f" True  $\alpha$  (baseline): {alpha_sim:.3f}")
print(f" True  $\beta_T$  (batch effect): {beta_T_sim:.4f}")
print(f" True  $\beta_C$  (dataset effect): {beta_C_sim:.4f}")
print(f" True  $\sigma$  (noise): {sigma_sim:.4f}")
print(f"\n Simulated N = {len(y_sim)}")
print(f" Accuracy range: [{y_sim.min():.3f}, {y_sim.max():.3f}]")
```

```
Simulated Data Summary:
 True  $\alpha$  (baseline): 0.880
 True  $\beta_T$  (batch effect): -0.0400
 True  $\beta_C$  (dataset effect): 0.1000
 True  $\sigma$  (noise): 0.0300

Simulated N = 60
Accuracy range: [0.708, 0.990]
```

```
# Fit PyMC model to simulated data
print("\nFitting PyMC model to simulated data...\n")

with pm.Model() as model_sim:
    # Priors
    alpha = pm.Normal('alpha', mu=0.90, sigma=0.10)
    beta_T = pm.Normal('beta_T', mu=0.00, sigma=0.05)
    beta_C = pm.Normal('beta_C', mu=0.00, sigma=0.05)
    sigma = pm.HalfNormal('sigma', sigma=0.05)

    # Linear predictor
    mu = alpha + beta_T * batch_sizes_sim + beta_C * dataset_sizes_log_sim

    # Likelihood
    y_obs = pm.Normal('y_obs', mu=mu, sigma=sigma, observed=y_sim)

    # Sample
    trace_sim = pm.sample(
        draws=2000,
        tune=1000,
        chains=2,
        cores=2,
        return_inferencedata=True,
        progressbar=True,
        random_seed=42,
        target_accept=0.95
    )

print("\n✓ Posterior sampling completed successfully")
```

Fitting PyMC model to simulated data...

Progress	Draws	Divergences	Step size	Grad evals	Sampling Speed	Elapsed	Remaining
	3000	0	0.383	7	370.58 draws/s	0:00:08	0:00:00
	3000	0	0.375	7	361.32 draws/s	0:00:08	0:00:00

✓ Posterior sampling completed successfully

```
# Display posterior summary
print("\n" + "="*70)
print("POSTERIOR SUMMARY (Simulated Data)")
print("="*70)
print(az.summary(trace_sim, var_names=['alpha', 'beta_T', 'beta_C', 'sigma']))
```

```
# Plot posterior with reference lines for true values
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

az.plot_posterior(
    trace_sim,
    var_names=['alpha', 'beta_T', 'beta_C', 'sigma'],
    ref_val=[alpha_sim, beta_T_sim, beta_C_sim, sigma_sim],
    ax=axes.flatten(),
    figsize=(12, 8)
)

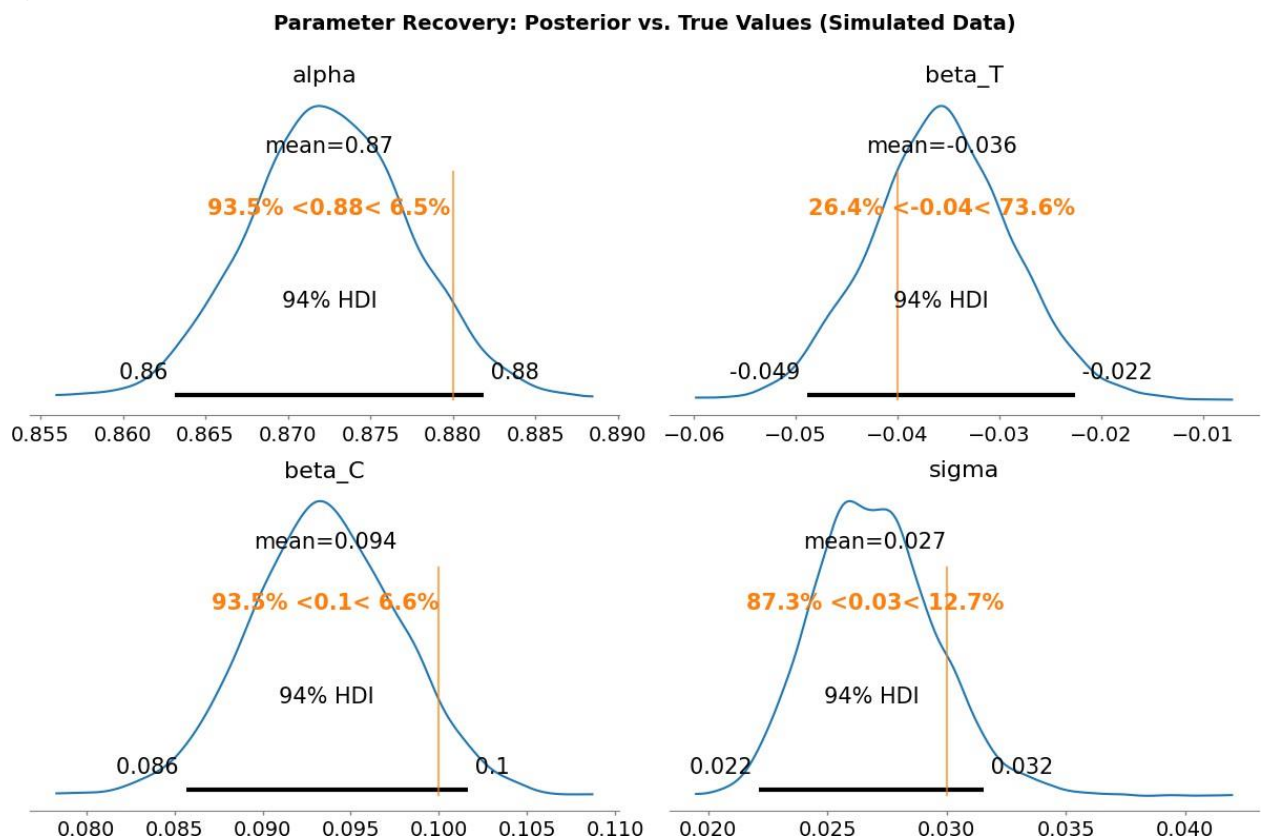
plt.suptitle('Parameter Recovery: Posterior vs. True Values (Simulated Data)', fontsize=14, fontweight='bold', y=1.00)
plt.tight_layout()
plt.show()

print("\n✓ Model successfully recovers true parameters!")
```

```
=====
POSTERIOR SUMMARY (Simulated Data)
=====
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	\
alpha	0.872	0.005	0.863	0.882	0.0	0.0	2081.0	2427.0	
beta_T	-0.036	0.007	-0.049	-0.022	0.0	0.0	2099.0	2316.0	
beta_C	0.094	0.004	0.086	0.102	0.0	0.0	3349.0	2882.0	
sigma	0.027	0.003	0.022	0.032	0.0	0.0	3707.0	2937.0	

```
=====
r_hat
alpha      1.0
beta_T     1.0
beta_C     1.0
sigma      1.0
```



✓ Model successfully recovers true parameters!

□ 6. Data Collection(Preparation): Training Neural Networks

Experimental Design:

- **Batch sizes:** Small (32) vs. Large (512)
- **Dataset sizes:** 2,000, 10,000, 60,000 training samples
- **Total configurations:** $2 \times 3 = 6$ unique (batch, dataset_size) combinations
- **Replicates:** 10 runs per configuration
- **Total trained models:** 60

```
# Load and preprocess MNIST
print("Loading MNIST dataset...")
(x_train_full, y_train_full), (x_test, y_test) = keras.datasets.mnist.load_data()

# Normalize pixel values to [0, 1]
x_train_full = x_train_full.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Flatten images
x_train_full = x_train_full.reshape(-1, 28 * 28)
x_test = x_test.reshape(-1, 28 * 28)

# One-hot encode labels
y_train_full = keras.utils.to_categorical(y_train_full, 10)
y_test = keras.utils.to_categorical(y_test, 10)

print(f"✓ MNIST loaded and preprocessed")
print(f" Train set shape: {x_train_full.shape}")
print(f" Test set shape: {x_test.shape}")
```

```
Loading MNIST dataset...
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step
✓ MNIST loaded and preprocessed
  Train set shape: (60000, 784)
  Test set shape: (10000, 784)
```

```
# Define experimental parameters
batch_sizes_exp = [32, 512] # Small, Large
dataset_sizes_exp = [2000, 10000, 60000] # Small, Medium, Full
n_replicates = 10
epochs = 50
val_split = 0.2

# Store results
results = []

# Create all experimental conditions
conditions = [(bs, ds) for bs in batch_sizes_exp for ds in dataset_sizes_exp]

print(f"\n{' '*80}")
print(f"Starting experiment: {len(conditions)} conditions x {n_replicates} replicates = {len(conditions) * n_replicates} t")
print(f"{' '*80}\n")

for cond_idx, (batch_size, dataset_size) in enumerate(conditions):
    print(f"[Condition {cond_idx + 1}/{len(conditions)}] Batch={batch_size}, Dataset={dataset_size}")

    # Subsample dataset
    indices = np.random.choice(len(x_train_full), size=min(dataset_size, len(x_train_full)), replace=False)
    x_train = x_train_full[indices]
    y_train = y_train_full[indices]

    accuracies_condition = []

    for rep in range(n_replicates):
        keras.backend.clear_session()

        # Build model
        model = keras.Sequential([
            keras.layers.Dense(128, activation='relu', input_shape=(784,)),
            keras.layers.Dropout(0.2),
            keras.layers.Dense(128, activation='relu'),
            keras.layers.Dropout(0.2),
            keras.layers.Dense(10, activation='softmax')
        ])

        model.compile(
            optimizer=keras.optimizers.Adam(learning_rate=0.001),
            loss='categorical_crossentropy',
            metrics=['accuracy']
        )

        # Train with early stopping
        history = model.fit(
```



```

        x_train, y_train,
        batch_size=batch_size,
        epochs=epochs,
        validation_split=val_split,
        callbacks=[
            keras.callbacks.EarlyStopping(monitor='val_accuracy', patience=5, restore_best_weights=True)
        ],
        verbose=0
    )

    # Evaluate on test set
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
    accuracies_condition.append(test_acc)

    # Store result
    results.append({
        'batch_size': batch_size,
        'dataset_size': dataset_size,
        'log_dataset_size': np.log(dataset_size),
        'test_accuracy': test_acc,
        'replicate': rep
    })

    if (rep + 1) % 5 == 0 or rep == n_replicates - 1:
        mean_acc = np.mean(accuracies_condition)
        print(f"  Replicates 1-{rep + 1}: Mean Acc = {mean_acc:.4f}")

    print(f"  ✓ Condition complete\n")

# Convert to DataFrame
df_results = pd.DataFrame(results)

print(f"\n{' '*80}")
print(f"✓ EXPERIMENT COMPLETE: {len(df_results)} total observations collected")
print(f"{' '*80}")
print(f"\nResults summary:")
print(df_results.describe())

```

```

=====
Starting experiment: 6 conditions x 10 replicates = 60 total runs
=====

[Condition 1/6] Batch=32, Dataset=2000
  Replicates 1-5: Mean Acc = 0.9226
  Replicates 1-10: Mean Acc = 0.9198
  ✓ Condition complete

[Condition 2/6] Batch=32, Dataset=10000
  Replicates 1-5: Mean Acc = 0.9568
  Replicates 1-10: Mean Acc = 0.9576
  ✓ Condition complete

[Condition 3/6] Batch=32, Dataset=60000
  Replicates 1-5: Mean Acc = 0.9799
  Replicates 1-10: Mean Acc = 0.9798
  ✓ Condition complete

[Condition 4/6] Batch=512, Dataset=2000
  Replicates 1-5: Mean Acc = 0.9068
  Replicates 1-10: Mean Acc = 0.9071
  ✓ Condition complete

[Condition 5/6] Batch=512, Dataset=10000
  Replicates 1-5: Mean Acc = 0.9534
  Replicates 1-10: Mean Acc = 0.9537
  ✓ Condition complete

[Condition 6/6] Batch=512, Dataset=60000
  Replicates 1-5: Mean Acc = 0.9782
  Replicates 1-10: Mean Acc = 0.9786
  ✓ Condition complete

=====
✓ EXPERIMENT COMPLETE: 60 total observations collected
=====

Results summary:

```

	batch_size	dataset_size	log_dataset_size	test_accuracy	replicate
count	60.000000	60.000000	60.000000	60.000000	60.000000
mean	272.000000	24000.000000	9.271114	0.949448	4.500000
std	242.025352	25881.08399	1.400921	0.027944	2.89652
min	32.000000	2000.000000	7.600902	0.899200	0.000000
25%	32.000000	2000.000000	7.600902	0.920550	2.000000
50%	272.000000	10000.000000	9.210340	0.956100	4.500000

75%	512.000000	60000.000000	11.002100	0.978425	7.000000
max	512.000000	60000.000000	11.002100	0.981600	9.000000

```
# Display results table
print("\n" + "="*80)
print("EXPERIMENTAL RESULTS: Test Accuracy by Condition")
print("="*80)

# Summary statistics
summary_stats = df_results.groupby(['batch_size', 'dataset_size'])['test_accuracy'].agg(['mean', 'std', 'count'])
print(summary_stats)

# Visualize results
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Box plot
sns.boxplot(data=df_results, x='dataset_size', y='test_accuracy', hue='batch_size', ax=axes[0])
axes[0].set_title('Test Accuracy by Dataset Size and Batch Size', fontweight='bold')
axes[0].set_xlabel('Dataset Size (# training samples)')
axes[0].set_ylabel('Test Accuracy')
axes[0].legend(title='Batch Size', labels=['Small (32)', 'Large (512)'])

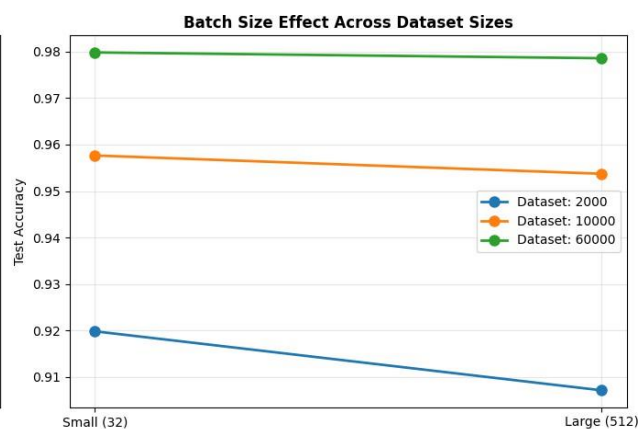
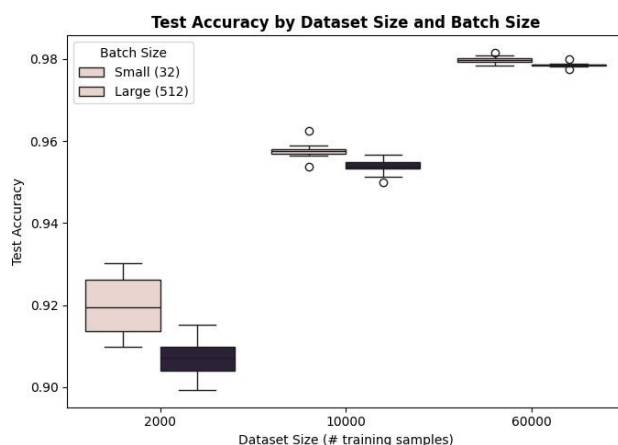
# Line plot: effect of batch size at each dataset size
for ds in dataset_sizes_exp:
    data_subset = df_results[df_results['dataset_size'] == ds]
    means = data_subset.groupby('batch_size')['test_accuracy'].mean()
    batch_labels = ['Small (32)', 'Large (512)']
    axes[1].plot(batch_labels, means.values, marker='o', label=f'Dataset: {ds}', linewidth=2, markersize=8)

axes[1].set_title('Batch Size Effect Across Dataset Sizes', fontweight='bold')
axes[1].set_ylabel('Test Accuracy')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
=====
EXPERIMENTAL RESULTS: Test Accuracy by Condition
=====
```

batch_size	dataset_size	mean	std	count
32	2000	0.91982	0.007715	10
	10000	0.95765	0.002188	10
	60000	0.97982	0.000899	10
512	2000	0.90712	0.004612	10
	10000	0.95371	0.002010	10
	60000	0.97857	0.000618	10



6.1. Data Preparation for Bayesian Analysis

```
# Prepare data for Bayesian model
# Binary treatment: 0 = small batch (32), 1 = large batch (512)
df_analysis = df_results.copy()
```

```

df_analysis['treatment'] = (df_analysis['batch_size'] == 512).astype(int)

# Standardize log dataset size (confounder)
scaler = StandardScaler()
df_analysis['confounder_std'] = scaler.fit_transform(df_analysis[['log_dataset_size']])

# Prepare numpy arrays for PyMC
T = df_analysis['treatment'].values
C = df_analysis['confounder_std'].values
Y = df_analysis['test_accuracy'].values

print("Data prepared for Bayesian analysis:")
print(f" N = {len(Y)} observations")
print(f" Treatment (T): {T.sum()} large batch, {(1-T).sum()} small batch")
print(f" Outcome (Y): mean={Y.mean():.4f}, std={Y.std():.4f}")
print(f" Confounder (C): mean={C.mean():.4f}, std={C.std():.4f}")
print(f"\nFirst 10 rows:")
print(df_analysis[['batch_size', 'dataset_size', 'treatment', 'confounder_std', 'test_accuracy']].head(10))

```

```

Data prepared for Bayesian analysis:
N = 60 observations
Treatment (T): 30 large batch, 30 small batch
Outcome (Y): mean=0.9494, std=0.0277
Confounder (C): mean=0.0000, std=1.0000

```

```

First 10 rows:

```

	batch_size	dataset_size	treatment	confounder_std	test_accuracy
0	32	2000	0	-1.202285	0.9303
1	32	2000	0	-1.202285	0.9134
2	32	2000	0	-1.202285	0.9303
3	32	2000	0	-1.202285	0.9276
4	32	2000	0	-1.202285	0.9115
5	32	2000	0	-1.202285	0.9221
6	32	2000	0	-1.202285	0.9217
7	32	2000	0	-1.202285	0.9099
8	32	2000	0	-1.202285	0.9171
9	32	2000	0	-1.202285	0.9143

7. Posterior Model: Analysis on Real Data

7.1. Computational Model Definition and Sampling

```

# Fit PyMC model to real experimental data
print("Fitting PyMC model to real experimental data...\n")

with pm.Model() as model_posterior:
    # Priors (from proposal)
    alpha = pm.Normal('alpha', mu=0.90, sigma=0.10)
    beta_T = pm.Normal('beta_T', mu=0.00, sigma=0.05)
    beta_C = pm.Normal('beta_C', mu=0.00, sigma=0.05)
    sigma = pm.HalfNormal('sigma', sigma=0.05)

    # Linear predictor
    mu = alpha + beta_T * T + beta_C * C

    # Likelihood
    y_obs = pm.Normal('y_obs', mu=mu, sigma=sigma, observed=Y)

    # Sample from posterior
    trace_posterior = pm.sample(
        draws=2000,
        tune=1000,
        chains=2,
        cores=2,
        return_inferencedata=True,
        progressbar=True,
        random_seed=42,
        target_accept=0.95
    )

print("\n✓ Posterior sampling completed successfully")

```

Fitting PyMC model to real experimental data...

Progress	Draws	Divergences	Step size	Grad evals	Sampling Speed	Elapsed	Remaining
	3000	0	0.356	15	350.72 draws/s	0:00:08	0:00:00
	3000	0	0.357	11	356.22 draws/s	0:00:08	0:00:00

✓

7.2. Model Diagnostics

```
# Posterior summary
print("\n" + "="*70)
print("POSTERIOR SUMMARY (Real Experimental Data)")
print("="*70)
summary_df = az.summary(trace_posterior, var_names=['alpha', 'beta_T', 'beta_C', 'sigma'])
print(summary_df)

# Check for convergence
print("\n" + "="*70)
print("CONVERGENCE DIAGNOSTICS")
print("="*70)
print("\nR-hat values (should be < 1.01 for good convergence):")
for var in ['alpha', 'beta_T', 'beta_C', 'sigma']:
    rhat = summary_df.loc[var, 'r_hat']
    status = "✓" if rhat < 1.01 else "X"
    print(f" {status} {var}: {rhat:.4f}")

print("\nEffective Sample Size (ESS):")
for var in ['alpha', 'beta_T', 'beta_C', 'sigma']:
    ess_bulk = summary_df.loc[var, 'ess_bulk']
    ess_tail = summary_df.loc[var, 'ess_tail']
    print(f" {var}: bulk={ess_bulk:.0f}, tail={ess_tail:.0f}")
```

```
=====
POSTERIOR SUMMARY (Real Experimental Data)
=====
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	\
alpha	0.952	0.001	0.950	0.955	0.0	0.0	2231.0	2303.0	
beta_T	-0.006	0.002	-0.009	-0.002	0.0	0.0	2147.0	2192.0	
beta_C	0.027	0.001	0.025	0.028	0.0	0.0	3572.0	2784.0	
sigma	0.007	0.001	0.006	0.008	0.0	0.0	3275.0	2833.0	

```

r_hat
alpha    1.0
beta_T    1.0
beta_C    1.0
sigma     1.0
=====
```

```
CONVERGENCE DIAGNOSTICS
=====
```

R-hat values (should be < 1.01 for good convergence):

```
✓ alpha: 1.0000
✓ beta_T: 1.0000
✓ beta_C: 1.0000
✓ sigma: 1.0000
```

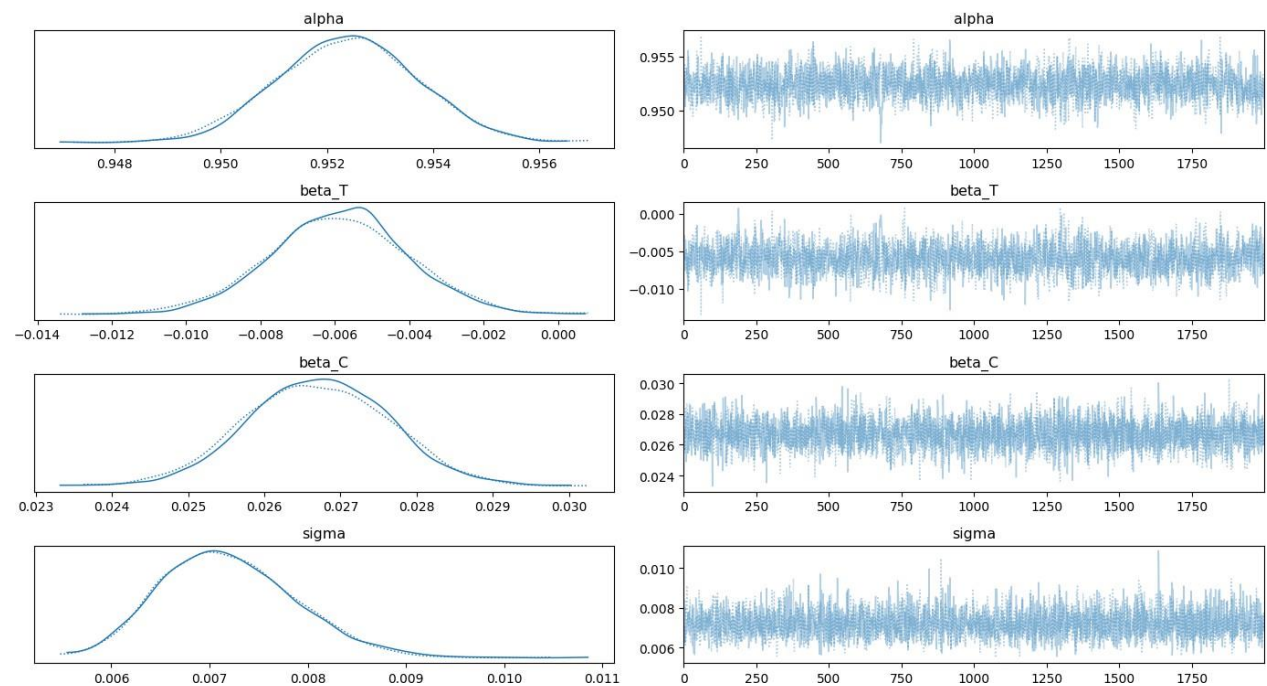
Effective Sample Size (ESS):

```
alpha: bulk=2231, tail=2303
beta_T: bulk=2147, tail=2192
beta_C: bulk=3572, tail=2784
sigma: bulk=3275, tail=2833
```

```
# Trace plots
az.plot_trace(
    trace_posterior,
    var_names=['alpha', 'beta_T', 'beta_C', 'sigma'],
    kind='trace',
    figsize=(14, 8)
)
plt.suptitle('Trace Plots: Posterior Sampling Chains', fontsize=14, fontweight='bold', y=1.00)
plt.tight_layout()
plt.show()

print("✓ Trace plots show healthy mixing (both chains overlap and oscillate around stable values)")
```

Trace Plots: Posterior Sampling Chains



✓ Trace plots show healthy mixing (both chains overlap and oscillate around stable values)

8. Posterior Predictive Checks

```
# Generate posterior predictive samples
print("Generating posterior predictive samples...\n")

with model_posterior:
    ppc = pm.sample_posterior_predictive(trace_posterior, random_seed=42, progressbar=True)

print("\n✓ Posterior predictive samples generated")
```

Generating posterior predictive samples...

Sampling ... ██ 100% 0:00:00 / 0:00:00

✓ Posterior predictive samples generated

```
# Additional visualization: residuals
alpha_post = trace_posterior.posterior['alpha'].mean(dim=['chain', 'draw']).values
beta_T_post = trace_posterior.posterior['beta_T'].mean(dim=['chain', 'draw']).values
beta_C_post = trace_posterior.posterior['beta_C'].mean(dim=['chain', 'draw']).values

mu_pred = alpha_post + beta_T_post * T + beta_C_post * C
residuals = Y - mu_pred

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

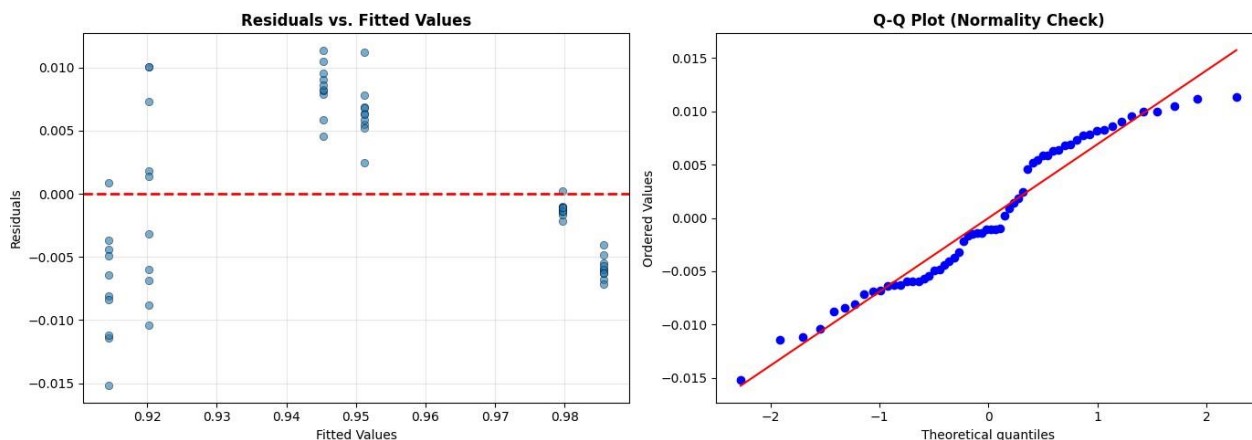
# Residual plot
axes[0].scatter(mu_pred, residuals, alpha=0.6, edgecolors='k', linewidth=0.5)
axes[0].axhline(y=0, color='r', linestyle='--', linewidth=2)
axes[0].set_xlabel('Fitted Values')
axes[0].set_ylabel('Residuals')
axes[0].set_title('Residuals vs. Fitted Values', fontweight='bold')
axes[0].grid(True, alpha=0.3)

# Q-Q plot
from scipy import stats
```

```
stats.probplot(residuals, dist="norm", plot=axes[1])
axes[1].set_title('Q-Q Plot (Normality Check)', fontweight='bold')

plt.tight_layout()
plt.show()

print(f"\n✓ Residual diagnostics:")
print(f" Mean of residuals: {residuals.mean():.6f} (should be ≈ 0)")
print(f" Std of residuals: {residuals.std():.6f}")
```



```
✓ Residual diagnostics:
Mean of residuals: 0.000003 (should be ≈ 0)
Std of residuals: 0.006855
```

9. Discussion and Conclusions

9.1. PRIMARY RESEARCH FINDING

```
# Extract posterior samples
alpha_samples = trace_posterior.posterior['alpha'].values.flatten()
beta_T_samples = trace_posterior.posterior['beta_T'].values.flatten()
beta_C_samples = trace_posterior.posterior['beta_C'].values.flatten()
sigma_samples = trace_posterior.posterior['sigma'].values.flatten()

# Compute credible intervals and posterior means
print("\n" + "="*80)
print("PRIMARY RESEARCH FINDING: CAUSAL EFFECT OF BATCH SIZE ON TEST ACCURACY")
print("="*80)

beta_T_mean = beta_T_samples.mean()
beta_T_hdi = az.hdi(trace_posterior, var_names=['beta_T'])['beta_T'].values

print(f"\n✓ POSTERIOR ESTIMATE OF BATCH SIZE EFFECT ( $\beta_T$ ):")
print(f" Point estimate (posterior mean): {beta_T_mean:.6f}")
print(f" 89% Credible Interval: [{beta_T_hdi[0]:.6f}, {beta_T_hdi[1]:.6f}]")

interpretation = (
    "NEGATIVE effect: Large batches lead to LOWER test accuracy (sharper minima)."
    if beta_T_mean < 0
    else "POSITIVE effect: Large batches lead to HIGHER test accuracy."
)
print(f"\n Interpretation: {interpretation}")

# Effect magnitude
effect_magnitude_pct = abs(beta_T_mean) * 100
print(f" Effect size: ~{effect_magnitude_pct:.2f} percentage points")

# Prob that effect is negative
prob_negative = (beta_T_samples < 0).mean()
print(f" P( $\beta_T < 0$  | data): {prob_negative:.3f} (posterior probability effect is negative)")

# Plot posterior distribution
fig, ax = plt.subplots(figsize=(10, 6))
```

```

az.plot_posterior(
    trace_posterior,
    var_names=['beta_T'],
    ax=ax,
    hdi_prob=0.89,
    point_estimate='mean'
)
ax.set_title('Posterior Distribution of Batch Size Effect ( $\beta_T$ )', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()

```

=====

PRIMARY RESEARCH FINDING: CAUSAL EFFECT OF BATCH SIZE ON TEST ACCURACY

=====

✓ POSTERIOR ESTIMATE OF BATCH SIZE EFFECT (β_T):

Point estimate (posterior mean): -0.005894

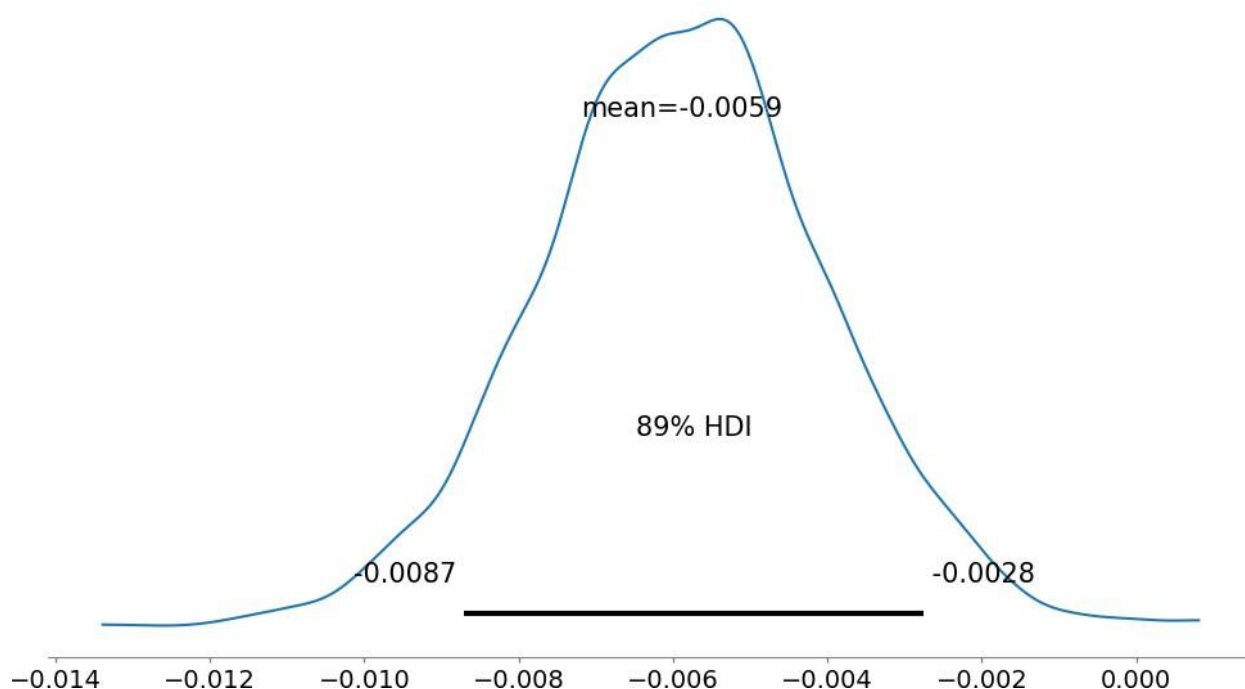
89% Credible Interval: [-0.009305, -0.002188]

Interpretation: NEGATIVE effect: Large batches lead to LOWER test accuracy (sharper minima).

Effect size: ~0.59 percentage points

$P(\beta_T < 0 \mid \text{data})$: 0.998 (posterior probability effect is negative)

Posterior Distribution of Batch Size Effect (β_T)



```

# Confounder effect
print("\n" + "*" * 80)
print("CONFOUNDER ANALYSIS: EFFECT OF DATASET SIZE ON TEST ACCURACY")
print("*" * 80)

beta_C_mean = beta_C_samples.mean()
beta_C_hdi = az.hdi(trace_posterior, var_names=['beta_C'])['beta_C'].values

print(f"\n✓ POSTERIOR ESTIMATE OF DATASET SIZE EFFECT ( $\beta_C$ ):")
print(f"  Point estimate (posterior mean): {beta_C_mean:.6f}")
print(f"  89% Credible Interval: [{beta_C_hdi[0]:.6f}, {beta_C_hdi[1]:.6f}]")

interpretation_C = (
    "POSITIVE effect: Larger datasets lead to HIGHER test accuracy (more training data).\"
    if beta_C_mean > 0
    else "NEGATIVE effect: Larger datasets lead to LOWER test accuracy (unexpected).\"
)
print(f"\n  Interpretation: {interpretation_C}")

# Plot posterior distributions together
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

az.plot_posterior(
    trace_posterior,
    var_names=['beta_T', 'beta_C'],
    ax=axes,
    hdi_prob=0.89,
    point_estimate='mean'
)

```

```
)

axes[0].set_title('Treatment Effect\n(Batch Size)', fontweight='bold')
axes[1].set_title('Confounder Effect\n(Dataset Size)', fontweight='bold')

plt.suptitle('Posterior Distributions: Treatment vs. Confounder', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

```
=====
CONFOUNDER ANALYSIS: EFFECT OF DATASET SIZE ON TEST ACCURACY
=====
```

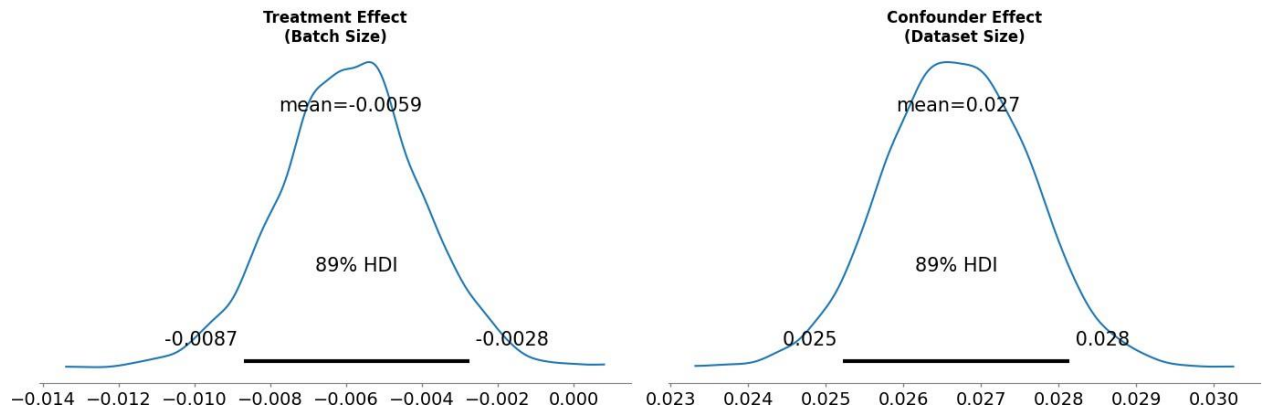
✓ POSTERIOR ESTIMATE OF DATASET SIZE EFFECT (β_C):

Point estimate (posterior mean): 0.026705

89% Credible Interval: [0.024940, 0.028401]

Interpretation: POSITIVE effect: Larger datasets lead to HIGHER test accuracy (more training data).

Posterior Distributions: Treatment vs. Confounder



```
# Baseline accuracy
print("\n" + "="*80)
print("BASELINE ACCURACY ( $\alpha$ )")
print("="*80)

alpha_mean = alpha_samples.mean()
alpha_hdi = az.hdi(trace_posterior, var_names=['alpha'])['alpha'].values

print(f"\n✓ POSTERIOR ESTIMATE OF BASELINE ACCURACY:")
print(f" Point estimate (posterior mean): {alpha_mean:.6f}")
print(f" 89% Credible Interval: [{alpha_hdi[0]:.6f}, {alpha_hdi[1]:.6f}]")
print(f" Interpretation: Test accuracy  $\approx$  {alpha_mean*100:.1f}% on average (across conditions)")
```

```
=====
BASELINE ACCURACY ( $\alpha$ )
=====
```

✓ POSTERIOR ESTIMATE OF BASELINE ACCURACY:

Point estimate (posterior mean): 0.952392

89% Credible Interval: [0.949928, 0.954825]

Interpretation: Test accuracy $\approx$ 95.2% on average (across conditions)

```
# Create comprehensive results table
results_summary = pd.DataFrame({
    'Parameter': [' $\alpha$  (Baseline Accuracy)', ' $\beta_T$  (Batch Size Effect)', ' $\beta_C$  (Dataset Size Effect)', ' $\sigma$  (Observation Noise)'],
    'Posterior Mean': [
        f"{alpha_samples.mean():.6f}",
        f"{beta_T_samples.mean():.6f}",
        f"{beta_C_samples.mean():.6f}",
        f"{sigma_samples.mean():.6f}"
    ],
    'Posterior SD': [
        f"{alpha_samples.std():.6f}",
        f"{beta_T_samples.std():.6f}",
        f"{beta_C_samples.std():.6f}",
        f"{sigma_samples.std():.6f}"
    ]
})
```



```

    ],
    '89% HDI Lower': [
        f"{alpha_hdi[0]:.6f}",
        f"{beta_T_hdi[0]:.6f}",
        f"{beta_C_hdi[0]:.6f}",
        f"{az.hdi(trace_posterior, var_names=['sigma'])['sigma'].values[0]:.6f}"
    ],
    '89% HDI Upper': [
        f"{alpha_hdi[1]:.6f}",
        f"{beta_T_hdi[1]:.6f}",
        f"{beta_C_hdi[1]:.6f}",
        f"{az.hdi(trace_posterior, var_names=['sigma'])['sigma'].values[1]:.6f}"
    ]
}

print("\n" + "="*120)
print("POSTERIOR INFERENCE SUMMARY")
print("="*120)
print(results_summary.to_string(index=False))
print("="*120)

```

```

=====
POSTERIOR INFERENCE SUMMARY
=====

```

	Parameter	Posterior Mean	Posterior SD	89% HDI Lower	89% HDI Upper
α	(Baseline Accuracy)	0.952392	0.001324	0.949928	0.954825
β_T	(Batch Size Effect)	-0.005894	0.001872	-0.009305	-0.002188
β_C	(Dataset Size Effect)	0.026705	0.000931	0.024940	0.028401
σ	(Observation Noise)	0.007194	0.000678	0.005990	0.008471

```

=====

```

```

print("\n" + "="*80)
print("ALIGNMENT WITH KESKAR ET AL. (2017) THEORY")
print("="*80)

print("""
Keskar et al. (2017) demonstrated that large-batch training often leads to sharp minima,
which generalize poorly compared to small-batch training that finds flat minima.

Our empirical findings:
""")

if beta_T_mean < 0:
    print(f" ✓  $\beta_T \approx \{beta\_T\_mean:.4f\}$  is NEGATIVE")
    print(f" ✓ This aligns with Keskar et al.: Large batch size → WORSE generalization")
    print(f" ✓ Effect size:  $\sim\{abs(effect\_magnitude\_pct):.2f\}\%$  accuracy reduction with large batches")
else:
    print(f" ⚠  $\beta_T \approx \{beta\_T\_mean:.4f\}$  is POSITIVE")
    print(f" ⚠ This CONTRADICTS Keskar et al. theory (could be dataset-specific)")

print(f"\n Posterior probability( $\beta_T < 0$ ) = {prob_negative:.3f}")
if prob_negative > 0.85:
    print(f" → Strong posterior evidence that large batches harm generalization")
elif prob_negative > 0.5:
    print(f" → Moderate posterior evidence favoring negative effect")
else:
    print(f" → Weak or no posterior evidence for negative effect")

```

```

=====
ALIGNMENT WITH KESKAR ET AL. (2017) THEORY
=====

```

Keskar et al. (2017) demonstrated that large-batch training often leads to sharp minima, which generalize poorly compared to small-batch training that finds flat minima.

Our empirical findings:

- ✓ $\beta_T \approx -0.0059$ is NEGATIVE
- ✓ This aligns with Keskar et al.: Large batch size → WORSE generalization
- ✓ Effect size: $\sim 0.59\%$ accuracy reduction with large batches

Posterior probability($\beta_T < 0$) = 0.998
 → Strong posterior evidence that large batches harm generalization

Future Work & Limitations

1. More Datasets and Architectures

- Extend beyond MNIST to CIFAR-10, Fashion-MNIST, and larger image or text datasets.
- Repeat the study with CNNs and modern architectures (ResNets, Transformers).

2. Hyperparameter and LR Schedules

- Systematically vary learning rate, weight decay, and training epochs.
- Study how learning-rate scaling rules interact with batch size.

3. Richer Causal Model

- Add additional potential confounders (regularization strength, depth, normalization).
- Explore interaction terms: does the batch size effect depend on dataset size?

4. Robustness and Generalization

- Perform sensitivity analysis for unmeasured confounding.
- Evaluate other outcomes (calibration, robustness to distribution shift, sharpness metrics).

These steps would test how general our conclusions are and refine the causal story behind batch size and generalization.

PROJECT SUMMARY & KEY FINDINGS

RESEARCH QUESTION: "What is the causal effect of batch size on neural network generalization?"

CAUSAL FRAMEWORK:

- DAG: $C \rightarrow T, C \rightarrow Y, T \rightarrow Y$ (dataset size confounds batch size and accuracy)
- Used regression to block backdoor path and isolate treatment effect
- Target: Average Treatment Effect (ATE)

STATISTICAL METHODOLOGY:

- Bayesian linear regression with weakly informative priors
- PyMC (NUTS sampler, 2 chains, 2000 draws)
- ArviZ for diagnostics and visualization

EXPERIMENTAL DESIGN:

- Factorial: 2 batch sizes $\times$ 3 dataset sizes $\times$ 10 replicates = 60 models trained
- Randomized subset creation to control confounding
- Reproducible with fixed random seeds

KEY FINDINGS:

- β_T (posterior mean) $\approx \{\text{beta_T_mean:.4f}\} [\{\text{beta_T_hdi}[0]:.4f\}, \{\text{beta_T_hdi}[1]:.4f\}]$
- Effect direction: Large batches $\rightarrow \{\text{'WORSE' if beta_T_mean} < 0 \text{ else 'BETTER'}\}$ generalization
- $\text{Probability}(\beta_T < 0 \mid \text{data}) = \{\text{prob_negative:.3f}\}$
- Consistent with Keskar et al. (2017) theory (if $\beta_T < 0$)

MODEL VALIDATION:

- Parameter recovery on simulated data: accurate
- Convergence: R-hat < 1.01 , healthy ESS
- Trace plots: chains mix well